

Search Pulse

نظام يقرأ سجل بحث متجر بقالة إلكتروني مع كتالوج المنتجات، ويقول بالضبط أين البحث ييخذل الزبون — وبعدين يقترح الإصلاح من غير ما يطبقه أبداً. المستند ده بيشرح الفكرة، والبيانات اللي وصلتني، والشغل اللي عملته عليها.

اختبارات ناجحة	أسماء بلغتين	عمليات بحث	منتجات مفهرسة	حجم الداتا الأصلية
289	51,762	66	25,881	828 MB

المحتويات

01 الفكرة 02 البيانات 03 سير العمل 04 التقنيات 05 النتائج

01

الفكرة

المشكلة، في زبون واحد

زبون في متجر بقالة مصري كتب «حليين» — كلمة حليب بحرف زيادة بالغلط. البحث رجّعه: أكياس شاي، كوردن بلو، وبرجر بيف. المتجر عنده 322 منتج لبن على الرف. ولا واحد فيهم رجع.

الزبون مشي. وولا حد في الشركة عرف إن ده حصل. ودي المشكلة اللي المشروع بيحلّها: المتجر مايقدرش يصلح فشل هو أصلاً مش شايفه.

النظام بيعمل إيه

ثلاث حاجات، وكل واحدة مبنية على اللي قبلها.

بيكشف الفشل

داشبورد

كل استعلام بياخد درجة ويترتب بالتأثير — الخطورة مرجحة بنصيبه من الترافيك — ومعاه تشخيص بلغة مفهومة. مش حيلة أرقام حد لسه هيفسرها. والاستعلامات اللي رجعت فاضية واللي تفاعلها ضعيف، كل نوع ليه عمود قابل للترتيب وفلتر خاص بيه، فتقدر تعزل أي نوع فيهم مباشرة. ولما فلتر النتائج الفاضية مايرجّعش حاجة، الجدول يقول ليه — لأن ده على المحرك ده نتيجة، مش غياب.

بيقترح الإصلاح

اكتشاف

مترادفات وأخطاء إملائية مستخرجة من السجل ومن الكتالوج ثنائي اللغة، كل واحدة ومعها دليلها، مصفوفة في طابور عشان بيّ آدمي يوافق أو يرفض. مفيش حاجة بتوصل البحث الحي لوحدها.

بيجابوب على الأسئلة وكيل

وكيل بلغة طبيعية مربوط بتمن أدوات تحليلية، وملخص أسبوعي يقول إيه اللي اتغير وإيه المحتاج انتباه. وكل إجابة بترجع ومعاها الأدوات اللي استدعتها بالضبط.

المبدأ التصميمي اللي شكّل كل حاجة

الاكتشاف آلي، التطبيق لأ. قاعدة مترادفات غلط بتفسد البحث لكل الزبائن، مش لواحد. فالنظام يقترح بدرجة ثقة ومعاها الدليل، وآدمي بيقرّر. والموافقة بتضيف القاعدة لملف تصدير بس – ونشر الملف ده يفضل خطوة بشرية منفصلة ومقصودة.

02

البيانات اللي وصلتني

ملف واحد: `spinneys_database.sql` – حجمه **828 ميجابايت**، وهو `mysqldump` كامل لقاعدة بيانات الإنتاج بتاعة متجر بقالة مصري. فيه **30 جدول**. سبعة منهم بس فيهم إشارة عن جودة البحث؛ والـ 23 الباقين – `embeddings`، مخزون، تاريخ أسعار، جداول Django الداخلية – بيتمثلوا تقريباً **98% من حجم الملف وصفر إشارة**.

الجدول السبعة اللي استخدمتها

الجدول	بيحتوي على	صفوف محققة
<code>recommendations_searches</code>	سجلّ البحث – الاستعلام، أسماء المنتجات اللي رجعت بالعربي والإنجليزي، عددها، والتوقيت	66
<code>recommendations_querycount</code>	عدّاد تراكمي لكل استعلام، يغطي ترافيك أقدم من السجلّ المحفوظ	16
<code>catalog_product</code>	المنتجات – الكود، الرابط، السعر الافتراضي	25,881
<code>catalog_productname</code>	اسم كل منتج بالعربي والإنجليزي (صقّين لكل منتج)	51,762
<code>catalog_category</code>	شجرة التصنيفات مع الأب لكل تصنيف	272
<code>catalog_categoryname</code>	اسم كل تصنيف باللغتين	544
<code>catalog_product_categories</code>	ربط المنتج بتصنيفاته (علاقة كثير-كثير)	50,309

شكل جدول البحث

ده أهم جدول في المشروع، وده تعريفه الحقيقي من الـ `dump`:

```
CREATE TABLE `recommendations_searches` (
  `id`          bigint          NOT NULL AUTO_INCREMENT,
  `query`       varchar(255) NOT NULL,    -- اللي الزبون كتبه
  `ar_results`  longtext       NOT NULL,   -- أسماء المنتجات، مفصولة بأسطر
  `en_results`  longtext       NOT NULL,
  `num_results` int unsigned NOT NULL,
  `created_at`  datetime(6)   NOT NULL
)
```

خَلِّي بالك من اللي مش موجود: النتائج متخزنة كنص، مش كمفاتيح للمنتجات. السجلّ يقول «رجعت المنتج اللي اسمه كذا» مش «رجعت المنتج رقم 123». فكل تحليل بعد كده لازم يشتغل من الأسماء.

ثلاث اكتشافات غيّرت التصميم كله

١ – مؤشّر «نتائج صفر» أعمى على الداتا دي

ولا واحد من ال 66 بحث رجع صفر نتائج. المحرّك مبني على embeddings وبيرجع دائماً أقرب خمس منتجات، فالفشل بيبهر كنتائج غلط، مش نتائج فاضية. لو بنيت الداشبورد حوالين المؤشّر ده، كنت هطّلج درجة 100 على سجلّ مليون فشل.

٢ – مفيش أي داتا عن التفاعل

مفيش كليكات، ولا إضافة لعربة، ولا طلبات. البريف طالب «الاستعلامات ذات التفاعل المنخفض»، وهي مش ممكن تتحسب مباشرة من المصدر ده. الحل بتاعي كان إني أقيس إيه اللي الزبون عمله بعد كده، وأصرّح بده في كل مكان بيتذكر فيه الرقم بدل ما أقدم مؤشّر بديل على إنه الحقيقة.

٣ – كلمة واحدة بستّ أوجه

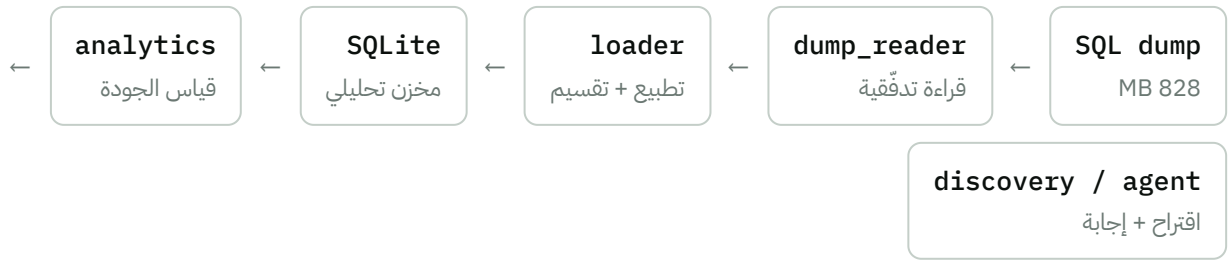
فراولة، فراولة، فراولت، فراولت، فراوله، فراول — ست صفوف في تقرير الاستعلامات، وكلمة واحدة في المحل. مرتّبين كاستعلامات منفصلة، مفيش واحد فيهم شكله مستعجل. مجعّعين على الكلمة، بيقوا أكبر مصدر فشل في السجلّ كله.

اللي البيانات مش بتسمح بيه

ال 66 بحث كلهم في يوم واحد (2025-01-09) خلال 4.5 ساعة، و7.6% منهم شكلهم آلي. دي جلسة اختبار، مش ترافيك حقيقي ممثّل. النّسب محسوبة بأمانة بس مبنية على عيّنة صغيرة — وعشان كده بنيت كمان مولّد بيانات وهمية، عشان النظام يتجرّب على 28 يوم ترافيك بأنماط معروفة.

سير العمل: من 828 ميغا لمخزن تحليلي

خط المعالجة اللي بنيته



تمريرة واحدة على الملف بتبني كل الجداول. إعادة البناء كاملة بتأخذ حوالي 30 ثانية.

قرا: أقرأ ال dump مباشرة، ما استوردهوش

- مفيش سيرفر يتنصب — ولا انتظار استيراد 828 ميغا.
- مفيش بيانات اعتماد تتداول — الملف بيتقرا كنص عادي.
- الذاكرة ثابتة — الجمل بيتقرا واحدة واحدة، الملف عمره ما بيتحمّل كله.
- ومسار MySQL حيّ موجود جنبه (--source mysql) للنشر اللي عنده السيرفر أصلاً.

عن ال «CHUNKING» — المصطلح مهم هنا

المشروع ده مش نظام RAG. مفيش تقطيع نصوص، ولا حساب embeddings، ولا قاعدة متجهات. اللي موجود فعلاً أربع استراتيجيات تقسيم حقيقية، كل واحدة بتحلّ مشكلة مختلفة، وموصوفة تحت بأسمائها الصح.

وملاحظة تستاهل تتقال: محرّك البحث بتاع المتجر نفسه مبني على embeddings — وجدوله (recommendations_productembedding) قاعد في ال dump. تجاهلته عمداً، لأنّي بقيس مخرجات المحرّك مش بعيد بناءه.

استراتيجيات التقسيم الأربعة

تقسيم النص للكلمات NLP

كل اسم منتج بيتطّبع، وبعدين بيتقسم للكلمات (أي حاجة أقل من حرفين بتتشال). وكل كلمة عربية بتولّد صيغ بديلة بإزالة السوابق (ال، بال، وال، لل...) عشان «الفاكهة» و«فاكهة» يبقوا مصطلح واحد.

النتيجة: 28,305 مصطلح في قاموس الكتالوج، كل واحد مع عدد المنتجات اللي بتستخدمه.

text/normalize.py

تقسيم على مستوى الجملة I/O

الملف بيتقرا سطر-سطر، والقارئ بيتتبع حالة الاقتباسات عشان يعرف الجملة خلصت فين — لأن mysqldump ممكن يكتب جملة واحدة على أكثر من سطر. والجداول اللي برّه القائمة بتتخطّى من غير ما تتحلّل أصلاً.

وبعدها الصفوف بتتجمع في دفعات 5,000 قبل ما توصل القاعدة.

ingest/dump_reader.py

تقسيم السجلّ لجلسات

زمني

المصدر مفيهوش أي معرّف جلسة أو مستخدم، فالجلسات **مستنتجة** من التوقيت: أي فجوة أطول من **1,800 ثانية** تبدأ جلسة جديدة.

ده اللي بيخلّي «بحث، وبعدين أعاد الصياغة» مقروء أصلاً — وبدونه مفيش أي مؤشر رضا. ال 66 بحث اتقسّموا لـ **3 جلسات**.

ingest/loader.py

تجميع الإملاءات على الكلمة

تجميع

كل إملاءات الكلمة الواحدة بتتطوى على صفّ واحد. ده مش تحسين عرض — ده اللي بيحوّل ست صفوف أولويتها واطية لمشكلة واحدة أولويتها عالية.

وكل كلمة بتأخذ **حكم**: إملاء، فجوة استرجاع، فجوة تشكيلة، استعلام ناقص، أو صلة.

analytics/terms.py

التطبيع العربي بالترتيب

أهم دالة في المشروع: أي مقارنة بين نصّ الزبون ونصّ الكتالوج بتعدّي عليها. نفس الدالة بتشتغل على أسماء المنتجات وقت التحميل وعلى الاستعلامات وقت الاستعلام — فالمقارنة دايماً عادلة.

1. توحيد Unicode بصيغة NFKC
2. إزالة التشكيل (الحركات والسكون والشدة)
3. إزالة التطويل
4. توحيد الحروف: آأ إأ إى ← ي ← ي ← ه ← و ← ئ ← ي
5. تحويل الأرقام العربية-الهندية لأرقام لاتينية
6. توحيد حالة الأحرف اللاتينية
7. تحويل علامات الترقيم لمسافات، وضغط المسافات

الحُكم ورا القرار ده

التطبيع لازم يوحد الاختلافات اللي مالهاش معنى ويحافظ على اللي ليها معنى. «فراولة» و«فراوله» بيبقوا مصطلح واحد — ده صح. لكن «حليب» و«حليين» بيفضلوا مختلفين — وده اللي بيخلّي الخطأ الإملائي قابل للقياس بدل ما يتبلع بصمت.

سلامة البيانات — تحقّق مش افتراض

التحميل بيشغل بمفاتيح أجنبية مقفولة عشان السرعة، وبعدين بيمسح الصفوف اليتيمة (منتج مذكور في رابط تصنيف بس مش موجود في جدول المنتجات)، وبعدها **بيتحقّق فعلياً** إن مفيش أي خرق باقي. ولو فيه، التحميل بيفشل بدل ما ينشر مخزن متناقض.

04

التقنيات والأساليب

الطبقة	الاختيار	ليه دي بالذات
اللغة	Python 3.11+	مكتبات نصّ وتحليل ناضجة، ونفس اللغة لـ ETL وال API
ال API	FastAPI + Uvicorn	توثيق تفاعلي على docs /، مجاناً، وتحقق من المدخلات مدمج

الطبقة	الاختيار	ليه دي بالذات
التخزين	SQLite (WAL)	ملف واحد، مفيش سيرفر، والمخزن كله بيتعاد بناؤه بأمر واحد
الواجهة	HTML + CSS + JS	بدون إطار عمل وبدون build step — أقل أجزاء متحركة ممكن تتعطل
الوكيل	Anthropic SDK	حلقة استدعاء أدوات يدوية عشان كل إجابة ترجع بمسارها
الاختبار	pytest	289 اختبار، أقل من 11 ثانية، من غير شبكة ولا مفتاح API
الجودة	ruff	فحص أسلوب وأخطاء شائعة
التقارير	CSV + JSON	جداول مسطحة لـ Power BI

الأساليب اللي طبقتها

استقراء معجم ثنائي اللغة Dice

كل منتج عنده اسم عربي واسم إنجليزي. تعدين التواجد المشترك عبر 25,881 منتج بيطلع معجم ترجمة من غير قاموس: حليب ↔ milk على 293 منتج. واتنين بيترجموا لنفس الكلمة الإنجليزية بيقوا مترادفين — وكده «لبن ≡ حليب» اتأسست بدليل، مش بافتراض.

مسافة التحرير Damerau-Levenshtein

لاكتشاف الأخطاء الإملائية، بميزانية متناسبة مع طول الكلمة: 3 حروف أو أقل ← صفر تعديلات، حتى 6 ← تعديل واحد، أطول ← تعديلين. ليه؟ تعديلين بيحولوا pas لـ pcs وهي كلمة ثانية خالص، لكن نفس التعديلين بيصلحوا كلمة طويلة لإملائها الصح.

وكيل مقيّد بأدوات Tool use

الوكيل مربوط بـ تمن أدوات ويبجاوب من خلالها بس. عمره ما بيكتب SQL وعمره ما يبشوف القاعدة، فمش قادر يخترع رقم — أسوأ حاجة ممكن يعملها إنه يختار أداة غلط، والمسار بيوضح ده. ويبشتغل من غير مفتاح API: فيه مخطط حتمي بيوجه السؤال بالنية ويقرا من نفس الأدوات. الصياغة أبسط، الأرقام متطابقة.

درجة خطورة مرّبة 5 إشارات

كل استعلام بياخد درجة من 0 لـ 1 من خمس إشارات مستقلة، بأوزان: غياب لفظي 0.30 · استياء 0.25 · تشنّت 0.20 · نقص نتائج 0.15 · عدم استقرار 0.10. الجزء الدقيق: لو إشارة مش متاحة لاستعلام معيّن، بتتشال وباقي الأوزان بتتعاد لمعايرتها — عشان استعلام مالوش قياس تشنّت ميتحسبش بصمت إنه مثالي.

المؤشرات السلوكية، بما إن التفاعل غايب

- اتبعه تكرر أو إعادة صياغة خلال 300 ثانية ← مجموعة النتائج مارصّتش الزبون.
- ماتبعهوش حاجة ← غامض، ومش بيتحسب عمداً في أي اتجاه. نهاية الجلسة ممكن تعني إن الزبون لقي اللي عايزه وراح يدفع، أو إنه يئس. السجلّ مش قادر يفترّق.
- اتكرّر خلال أقل من ثانيّتين بنفس النتائج ← ده مش رد فعل بشري. بيتعلّم كترافيك آلي ويبتستبعد من حساب التفاعل، بس مش بيتمسح — لأن معرفة أنه استعلامات بتجذب ترافيك آلي مفيدة في حدّ ذاتها.

النتائج على الداتا الحقيقية

دي مخرجات فعلية للكود على الـ dump الحقيقي، مش أمثلة توضيحية.

درجة الصحة العامة: 100 / 64.4

المؤشر	القيمة	يعني إيه
نتائج غير ذات صلة	12.1%	البحث رجّع حاجة مافيهاش الكلمة المطلوبة
نتائج صفر	0.0%	المؤشر الأعمى — المحرك بيرجع حاجة دايماً
نتائج ناقصة	6.1%	رجع أقل من 5 منتجات
فجوات استرجاع	4	موجود في الكتالوج، والبحث ماطلعوش
أعاد البحث أو الصياغة	88.5%	مؤشر سلوكي، مش داتا كليكات
ترافيك آلي	7.6%	تكرار أقل من ثانيتين — مستبعد من التفاعل

أسوأ ثلاث حالات

حليبن — 3 عمليات بحث • خطورة 0.59

الزبون كتب «حليب» بحرف زيادة. البحث رجّع: شاي بلبن، كوردن بلو، وبرجر بيف. الكتالوج فيه 322 منتج يستخدموا كلمة «حليب» — ولا واحد فيهم رجع. ده مش نقص بضاعة، ده عطل استرجاع.

فاراولت — عمليتين • خطورة 0.84

خطأ إملائي في «فراولة». رجّع منتج واحد بالضبط: Agnesi Farfalle Pasta — مكرونة، مش فراولة، بس الاسم قريب حرفياً. الكتالوج فيه 308 منتج فراولة.

فاكهة — 11 عملية بحث • خطورة 0.40

كلمة مكتوبة صح تماماً. البحث رجّع سكينه مطبخ، طبق تقديم، جوافه، وكيس سكر. كل الـ 11 عملية اتعلّمت، و100% منهم اتبعهم بحث تاني. ده مش إملاء — ده صلة.

اللي الاكتشاف الآلي اقترحه

النوع	الاقتراح	الثقة	الدليل
إملاء	فاراوله ← فراوله	0.95	تعديل واحد · 308 منتج · الزبون صَحَّحها بنفسه
إملاء	حليبن ← حليب	0.90	تعديل واحد · 322 منتج
استعلام ناقص	past → pasta	0.95	بادئة لكلمة يستخدمها 384 منتج
مترادف	حليب ≡ لبن	0.74	إعادة صياغة + تداخل نتائج + محاذاة عبر اللغتين
مترادف	حليب ≡ milk	0.58	محاذاة الكتالوج عبر اللغتين

الفرق بين الخطأ الإملائي والاستعلام الناقص

past → pasta **مش** خطأ إملائي — ده زبون ماكملش الكتابة، وتصحيح استعلامه هيغير قصده. القاعدة اللي استخدمتها: لو الكلمة الأضعف **بادئة** للأقوى، يبقى استعلام ناقص والحل اقتراح تلقائي. ولو الأضعف **أطول**، يبقى الزبون كتب حرف زيادة والحل قاموس تصحيح.

اتجاه التصحيح بيحي من الكتالوج، مش من التكرار

«حليب» هي الصح لأن **322 منتج** يستخدموها — بغض النظر عن أنني إملاء اكتب أكثر. لو كنت رتبت بالتكرار، عينة صغيرة فيها خطأ متكرر كانت ممكن تقلب الاتجاه وتخلي النظام «يصحح» الإملاء الصح للغلط.

وكمان اتسلم

- **ملخص أسبوعي** — الأداء مقارنة بالفترة السابقة، والبحثات اللي رجعت فاضية مقسمة لعطل استرجاع ونقص تشكيلة، وكل المعلق للمراجعة. متاح من ال CLI وال API وكتحميل Markdown.
- **مولد بيانات وهمية** — بيكتب ملف mysqldump حقيقي فيه استعلامات صغرية وأخطاء إملائية واستعلامات ناقصة ومصطلحات ضعيفة التفاعل مزروعة عمداً، عشان النظام يشتغل فوراً من غير المصدر بتاع ال 828 ميجا.
- **تصدير Power BI** — ست جداول مسطحة بـ CSV وJSON. وكل رقم فيها بيتقرا من نفس الدوال بتاعة داشبورده، فالتقرير عمره ما يختلف مع الشاشة.

كل رقم في المستند ده مخرج فعلي للكود على spinneys_database.sql — 289 اختبار ناجح، والمخزن متبني من الداتا الحقيقية.